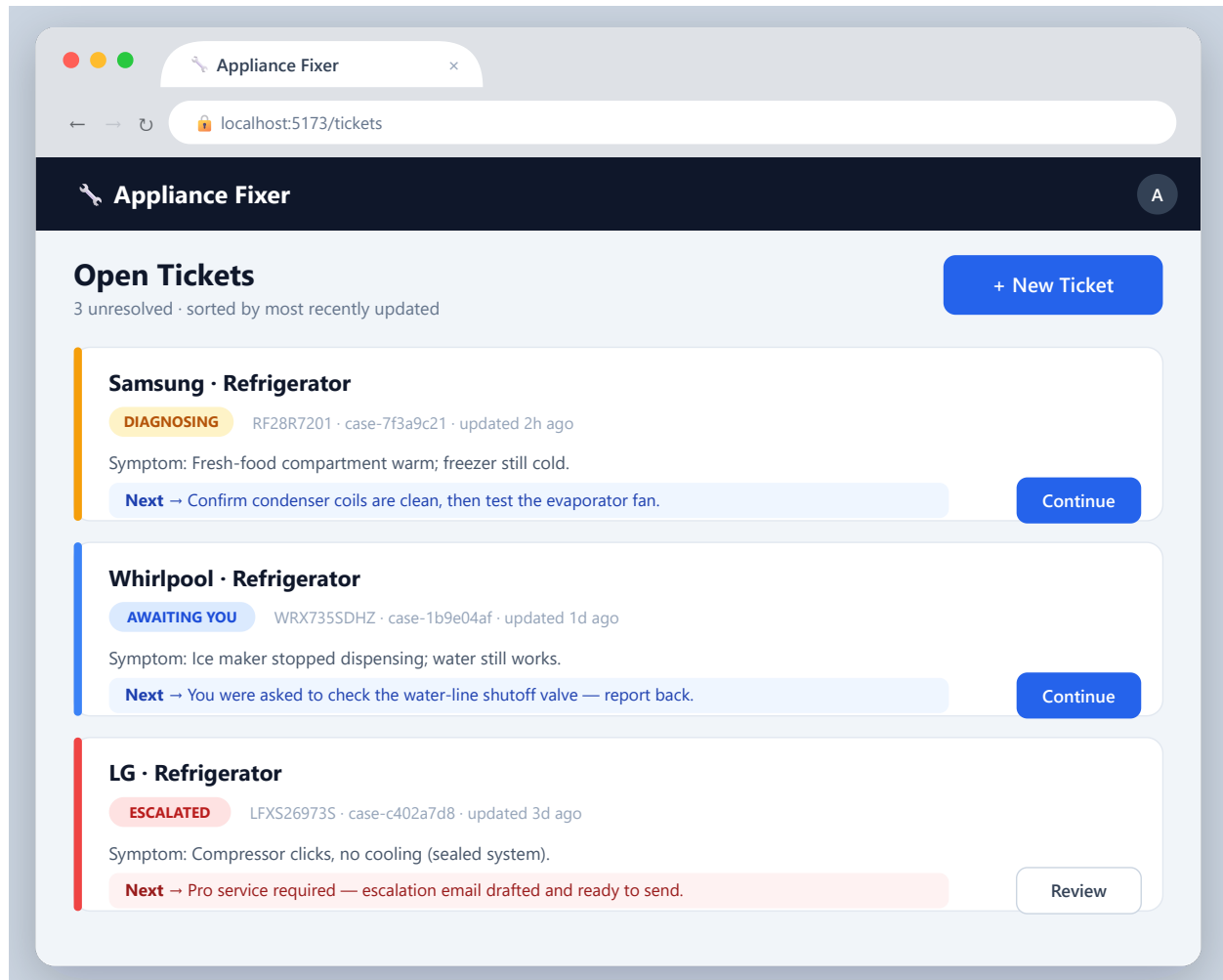


CONTENTS

Frontend Design: Appliance Fixer — Ticket Console	
1. Purpose	
2. How this maps to what already exists	
3. Dashboard — the home screen	
4. New Ticket flow	
5. Backend: the new /api/tickets layer	
6. Computing “next steps” (the headline detail)	
7. Ticket detail	
8. Architecture	
9. Suggested tech & build order	
10. Out of scope (v1)	
11. Open questions	

Frontend Design: Appliance Fixer — Ticket Console

Status: PROPOSED · 2026-06-21 Owner: (you) · Backend: existing Google ADK agent (`appliance_fixer.root_agent`) Related: [DESIGN.md](#), [IMPLEMENTATION PLAN.md](#)



1. Purpose

The agent today is reachable only through ADK's generic developer playground (`adk web`), which is a single ephemeral chat with no concept of “my open problems.” This document designs a purpose-built **Ticket Console** frontend with two headline requirements:

1. A prominent “+ **New Ticket**” button that starts a new appliance-problem case.
2. An **at-a-glance list of all unresolved tickets**, each showing a brief “**next steps**” summary so the user knows what to do without reopening the chat.

A “ticket” is exactly the agent’s existing **case** — this frontend is a view over the `CaseStore` that already exists; it does not invent a new data model.

2. How this maps to what already exists

Frontend concept	Backend reality (today)
Ticket	A row in the <code>cases</code> table (case_store.py)
Ticket ID	<code>case_id</code> (e.g. <code>case-7f3a9c21</code>)
Unresolved	<code>status</code> $\in$ <code>intake</code> , <code>diagnosing</code> , <code>awaiting_user</code> , <code>escalated</code> (everything except <code>resolved</code>)
Title	<code>brand</code> + <code>model_number</code> + <code>appliance</code>
Symptom	<code>data.symptom_text</code>
Steps taken	<code>data.steps[]</code> with per-step <code>outcome</code>
Next steps	Derived — see §6
The chat	ADK conversation via <code>/run_sse</code> , reopened with <code>reopen_existing_case(case_id)</code>

The one real gap: the running server ([fast_api_app.py](#)) exposes ADK’s chat/session routes but **no endpoint to *list* cases**. The dashboard cannot be built without a thin REST layer over `CaseStore` (§5). This is the bulk of the new backend work; the chat itself is reused as-is.

3. Dashboard — the home screen

The landing screen. Unresolved tickets are the entire focus; “+ New Ticket” is the only primary action.


Appliance Fixer
A

Open Tickets

3 unresolved · sorted by most recently updated

[+ New Ticket](#)

Samsung · Refrigerator
DIAGNOSING
RF28R7201 · case-7f3a9c21 · updated 2h ago

Symptom: Fresh-food compartment warm; freezer still cold.

Next → Confirm condenser coils are clean, then test the evaporator fan.

[Continue](#)

Whirlpool · Refrigerator
AWAITING YOU
WRX735SDHZ · case-1b9e04af · updated 1d ago

Symptom: Ice maker stopped dispensing; water works.

Next → You were asked to check the water-line shutoff valve — report back.

[Continue](#)

LG · Refrigerator
ESCALATED
LFXS26973S · case-c402a7d8 · updated 3d ago

Symptom: Compressor clicks, no cooling (sealed system).

Next → Pro service required — escalation email drafted and ready to send.

[Review](#)

Anatomy

- **Header bar** — product name + account.
- **”+ New Ticket”** — top-right primary button; opens the modal in §4.
- **Ticket cards**, newest-updated first. Each card shows:
 - Appliance title (`brand · appliance`) and a **status badge** (color-coded).
 - A muted meta line: `model_number · case_id ·` relative “updated” time (from `updated_at`).
 - **Symptom** line (`data.symptom_text` , truncated).
 - A highlighted `Next →` strip: the one-line “what to do next” summary (§6).
 - A **Continue** button (→ ticket detail, §7). Escalated tickets show **Review** instead.
- **Resolved tickets are hidden** behind a “View resolved (n)” link so the open list stays the focus.

Status → color

Status	Badge	Meaning
<code>intake</code>	grey	Created, still gathering brand/model/symptom
<code>diagnosing</code>	amber	Actively working a fix
<code>awaiting_user</code>	blue	Waiting on the user to try something / report back
<code>escalated</code>	red	Handed to a pro; escalation email drafted
<code>resolved</code>	green	Closed (hidden from the open list)

4. New Ticket flow

The button opens a focused modal. The goal is the smallest form that lets the agent start diagnosing: appliance + brand + model + symptom, with an optional spec-plate photo that auto-fills brand/model/error-code via the existing `read_spec_plate` tool.

Appliance Fixer

New Ticket

Describe what's wrong. Add a photo of the spec plate to auto-fill.

Appliance type

Refrigerator

Brand **Model number**

e.g. Samsung RF28R7201...

Scan spec plate — auto-fills brand, model & error code

What's wrong?

Fresh-food side is warm but the freezer is still cold...

Error code (optional)

e.g. 22 E

Cancel Create

Fields

- **Appliance type** — defaults to *Refrigerator* (the agent's primary focus).
- **Brand** and **Model number** — free text; validated server-side via `verify_model_number`.
- **Scan spec plate** — upload a photo; backend runs `read_spec_plate` and pre-fills brand/model/error code (the user can correct them). Reuses the validated plate-read path.
- **What's wrong?** — the symptom (`symptom_text`).
- **Error code (optional)** — if a code is on the display.

On Create: `POST /api/tickets` → calls `initialize_new_case(...)`, which returns a `case_id` with `status = diagnosing`. The frontend navigates straight into the ticket detail (§7) so the user can immediately start the conversation. New ticket then appears

on the dashboard automatically.

Edge cases (already handled by the agent, surfaced in the UI):

- Unreadable plate → “couldn’t read it — retake or type the model” (retry ×2 then manual).
- Valid-but-wrong model → field shows a confirm/retake prompt.
- Dangerous symptom (gas/refrigerant/mains) → ticket is created but the agent’s `before_model_callback` safety guard steers it toward escalation.

5. Backend: the new `/api/tickets` layer

A thin FastAPI router added to [fast_api_app.py](#), wrapping the existing `CaseStore`. No new persistence — same SQLite DB the agent already writes.

```
GET /api/tickets?status=open      -> [ TicketSummary ]    # dashboard list
GET /api/tickets/{case_id}       -> TicketDetail           # detail panel
POST /api/tickets                -> { case_id }              # New Ticket ->
initialize_new_case
POST /api/tickets/{case_id}/plate -> { brand, model, error_code } #
read_spec_plate
```

`TicketSummary` (one card):

```
{
  "case_id": "case-7f3a9c21",
  "title": "Samsung · Refrigerator",
  "model_number": "RF28R7201",
  "status": "diagnosing",
  "symptom": "Fresh-food compartment warm; freezer cold.",
  "next_step": "Confirm condenser coils are clean, then test the evaporator fan.",
  "updated_at": "2026-06-21T18:04:00Z"
}
```

The chat itself is **not** re-implemented — the detail view talks to ADK’s existing `/run_sse`, opening the case with `reopen_existing_case(case_id)` so the agent loads the full recap and prior steps. Two stores stay separate, exactly as the agent already

intends: `CaseStore` = the structured ticket; ADK session = the live conversation.

6. Computing “next steps” (the headline detail)

The dashboard’s value is the one-line next-step per ticket. Three derivation tiers, cheapest first — **no extra LLM call on dashboard load**:

1. **Escalated** → `"Pro service required – escalation email drafted and ready to send."` (from `data.escalation`).
2. **Has a pending/last step** → surface the most recent not-yet-resolved step's instruction (from `data.steps[]`), e.g. *"You were asked to check the water-line shutoff — report back."*
3. **Diagnosing with a hypothesis** → first un-tried fix from `get_fixes(...)` against the stored `diagnosis`, e.g. *"Confirm coils are clean, then test the evaporator fan."*

Persist this as a `next_step` string on the case (write it whenever `record_step_result` or `lookup_fixes` runs) so the list is a pure DB read and stays snappy — consistent with the project’s existing caching decision (cache expensive reads in the case `data`).

7. Ticket detail

Opened from **Continue/Review**. Left: the structured case (read from `CaseStore`). Right: the live agent chat (ADK `/run_sse`). The summary panel makes the agent’s memory visible — symptom, diagnosis, every step + outcome, the next step, and an **Escalate to a pro** action.


Appliance Fixer

[← All tickets](#)

Samsung · Refrigerator

RF28R7201 · case-7f3a9c21

Case summary

SYMPTOM
Fresh-food warm; freezer cold.

DIAGNOSIS
Restricted airflow / dirty coils
(confidence: medium)

STEPS TAKEN

- 1. Set temp to 37°F — no change
- 2. Cleared vents — no change
- 3. Clean condenser coils — pending

NEXT STEP

Confirm coils are clean, then test the evaporator fan motor.

Escalate to a pro

DIAGNOSING

Coils on the back can collect dust and block cooling. Can you vacuum them?

Done — they were pretty dusty.

Great. Give it 2–3 hours to recover. If it's still warm, the evaporator fan may be failing — we'll test that next.

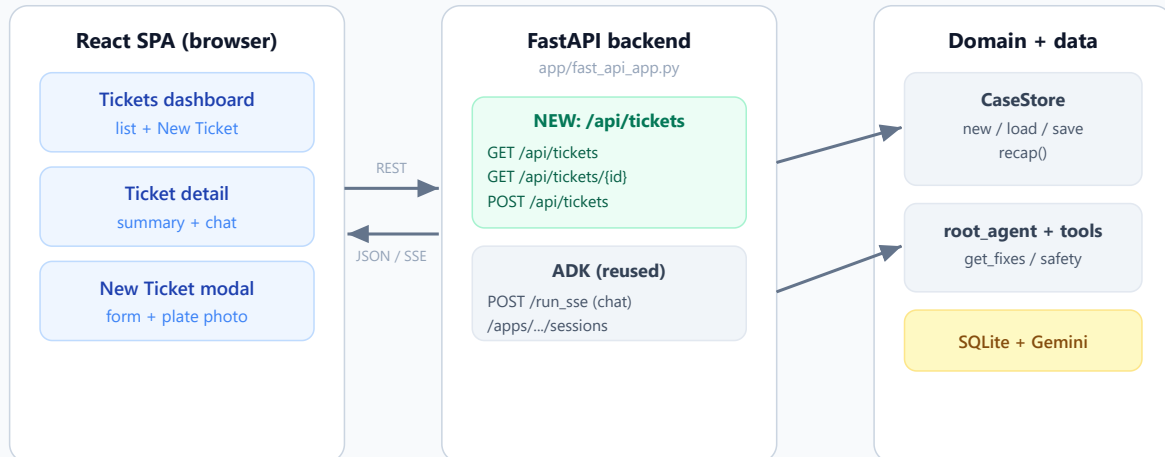
⚠ I won't advise sealed-system or mains-voltage work — that needs a pro.

Type a reply...

- **Case summary (left)** — `symptom_text`, `diagnosis` (hypothesis + confidence), `steps[]` rendered as a checklist (green = resolved/done, amber = pending), the derived next step, and an escalate button (→ `generate_escalation_draft`, draft-only).
- **Chat (right)** — the reopened conversation; photo attach for symptom/plate photos. The safety guard's refusals render inline as a distinct warning bubble, so “this needs a pro” is visible rather than buried.

8. Architecture

System Architecture — Ticket Frontend over the existing ADK agent



The dashboard's ticket list and "next steps" are served by a thin `/api/tickets` layer reading **CaseStore**; the chat reuses ADK's existing `/run_sse`.
Green = new code to write. Grey = already exists today.

Green = new code (the `/api/tickets` router + the React SPA). Grey = reused as-is (`CaseStore` , `root_agent` , tools, ADK `/run_sse` , SQLite, Gemini). The frontend is a read/write skin over machinery that already exists and is already tested.

9. Suggested tech & build order

- **Stack:** React + Vite + TypeScript SPA, served as static files by the same FastAPI app (or standalone in dev). Keeps one deployable, reuses ADK's CORS config.
- **Build order**
 1. `GET /api/tickets` + `GET /api/tickets/{id}` over `CaseStore` (+ tests). Unblocks everything.
 2. Dashboard (list + status badges + next-step strip).
 3. New Ticket modal → `POST /api/tickets` (+ optional `/plate`).
 4. Ticket detail: summary panel, then wire chat to `/run_sse` via `reopen_existing_case` .
 5. Persist the derived `next_step` ; add the resolved-tickets view.

10. Out of scope (v1)

- Multi-user accounts / auth (the agent uses a single `user-default` today).
- Real outbound email — escalation stays **draft-only**, matching the agent.
- Appliances beyond refrigerator (washer is a gated stretch in the agent itself).
- Editing/deleting tickets, search/filter beyond open-vs-resolved, notifications.

11. Open questions

1. `next_step` **freshness** — persist-on-write (§6, recommended) vs. compute live per request? Persist-on-write is snappier but can lag if the agent changes state outside the tools.
2. `awaiting_user` **status** — the agent currently only sets `diagnosing` / `resolved` ([agent.py:185](#)). To light up the blue “Awaiting you” badge, `record_step_result` should set `awaiting_user` when it logs a step whose `outcome` is `unsure` / pending. Small agent tweak — in scope or defer?
3. **Auth boundary** — fine to keep single-user for the capstone demo, or stub a user id?